Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kapsamı ve kaynakları (ekrandan, 00:10:00 ve 00:14:50)
  - Ders iki kısımdan oluşur: Kısım 1 Gang of Four (Dörtlü Çete) tasarım kalıpları, Kısım 2 Code Smells and Refactoring (birim sınamaları ile).
  - Dersin amacı, nesneye dayalı tasarımın nasıl doğru yapılacağını öğrenmek ve bunu uygulama becerisi kazanmaktır. Tasarım kalıpları işin doğru yapılışını, kod kusurları (code smells) ise yanlış kullanım örneklerini adlandırarak gösterir.
  - Refactoring, yapılmış bir tasarım hatasını geriye doğru uyumluluğu koruyarak düzeltmektir. Kaynak kitaptaki bütün kod kusurları değil, birbiriyle en çok bağlantılı olan bir seçme küme işlenecektir.
  - Kısım 1'in temel kaynağı: *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994.
  - Ek kaynaklar (slayttaki liste): *Design Patterns Explained* (Shalloway, Trott), *Head First Design Patterns* (Freeman vd.), *Design Patterns* (Lasater), *Design Patterns Java Workbook* (Metsker), *Applied Java Patterns* (Stelting, Maassen). Hocanın kalıp örneklerinin bir kısmı bu kitaplardan doğrudan ya da uyarlanarak alınmıştır.
  - Kısım 2'nin kaynağı: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999.
- Kalıpların derste işleniş biçimi
  - Önce kalıp tanıtılır, sonra bir probleme en az iki farklı biçimde uyarlanır: ilki probleme kafadan gelen, "bugünü kurtaran" çözümdür; küçük bir değişiklik istendiğinde bu çözümün nasıl sıkıntıya girdiği gösterilir.
  - Ardından kalıp uygulanır, kazanılan avantajlar ve ortaya çıkan dezavantajlar tartışılır. Gang of Four kitabı da her kalıptan sonra avantaj, dezavantaj ve tartışma bölümü verir; "sihirli değnek" çözüm yoktur.
- Ön koşullar (ekrandan, 00:14:50)
  - Herhangi bir güncel nesneye yönelik programlama diline hakimiyet (C++, Java, C# gibi). Kod örnekleri Java ile verilir; konu dile özgü bir özelliğe dayanmaz.
  - Kalıtım ve çok biçimlilik neredeyse bütün kalıplarda kullanılır; en temel gereksinim budur.
  - UML sınıf şemalarına hakimiyet. Önerilen kaynaklar: *UML Distilled* (Martin Fowler, 3. baskı, 2003) ve *UML 2.0 in a Nutshell* (Pilone, Pitman).
  - UML standardı Gang of Four kitabından sonra oturmuştur. Kitaptaki şemalar eski gösterimle çizilmiştir; derste yeni (güncel UML) gösterim kullanılır.
- Yazılım kalitesi ve iyi tasarım (ekrandan, 00:40:00)
  - İyi tasarım tanımı: problem alanı hangi düzeyde parçalara ayrılabiliyorsa o düzeyde parçalara ayrılabilen tasarım "iyi" sayılır.
  - Dış kalite özellikleri kullanıcıya yöneliktir: doğruluk, etkinlik, kullanım kolaylığı.
  - İç kalite özellikleri programcıya yöneliktir: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı.
- Neden iyi tasarım? (ekrandan, 00:51:30)
  - Yazılım projeleri önemli oranda başarısız olur. Etken çoktur, ama "tasarımın iyiliği" denetlenebilen bir etkendir.
  - Hatayı düzeltmenin bedeli hayat döngüsünde ilerledikçe üstel olarak artar; çabanın çoğu bakım aşamasında harcanır.

  - Katı başarı tanımı: gereksinimlerin tamamı, belirlenen süre ve belirlenen bütçe içinde karşılanmalıdır. Bu tanımla başarı oranı yaklaşık %50'dir. Üç ayaktan birinde sapmaya izin veren "zorlanılmış" (esnetilmiş) başarı tanımıyla oran %80'lere çıkar.
- Bağlaşım (coupling) ve uyum (cohesion) (ekrandan, 01:07:30 ve 01:30:00)
  - Amaç düşük bağlaşım (low coupling) ve yüksek uyumdur (high cohesion). İlgi alanlarının ayrılması ilkesi bu ikisine uyulduğunda kendiliğinden sağlanır.
  - Bağlaşım, bir parçanın (sınıf, alt sistem, paket) diğer parçalarla ilişkide olma oranıdır.
  - Uyum, bir parçanın sorumluluklarının birbiriyle uyumlu olma oranıdır.
- Bağlaşım ve uyumun sayısal örnekleri (ekrandan, 01:20:00, 01:23:40, 01:40:00, 01:50:00)
  - A→B→C zinciri ile A→B, A→C yıldızı karşılaştırıldı: toplam, maksimum ve dolaylı bağlaşım ölçütleri farklı sonuçlar verir.
  - Beş metot ve dört alanlı bir sınıfta uyum ölçüldü, sınıf iki sorumluluğa bölünerek uyum artırıldı.
  - Veritabanı işlemleri ayrı bir sınıfa (D) verildiğinde bağlaşım artar, ama uyum artar ve katmanlı bir çözüm ortaya çıkar.
- Tasarım kalıbı kavramı ve akademik kaynak arama
  - "Tasarım kalıbı" terimi mimarlıktan ödünç alınmıştır. Sık karşılaşılan modelleme problemlerinin iyi çözümüne yönelik önerilerdir.
  - Kalıplar üç kategoride gruplanır: yaratımsal (creational), yapısal (structural), davranışsal (behavioral).
  - Bilgisayar mühendisliği için önemli tam metin veri tabanları: ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, Wiley; Web of Science özet araması yapar.

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlaşım ilişkinin niteliğine değil, varlığına ve sayısına bakar
  - İki sınıf arasında herhangi bir ok varsa bağlaşım vardır. Ok yönlüdür ve bağımlılık yönünü gösterir; çift yönlü ilişki iki kez sayılır.
  - A'dan B'ye 1 ya da * nicelikli ilişki olması bağlaşımı değiştirmez.
- Kalıtım "ağır sıklet" bir ilişkidir
  - Kalıtım kurulduktan sonra değiştirilemez: bir Çalışan nesnesi emekli olunca Kişi nesnesine dönüşemez, yeni bir nesne (yeni kimlik) oluşturmak gerekir. Sahiplik ilişkisinde ise B1 nesnesinin yerine B2 konabilir.
  - Bu yüzden bağlaşım hesabında kalıtım 1,5 ya da 2 gibi bir katsayıyla sayılabilir.
  - Tasarım kalıplarındaki hatalı çözümlerde işler genelde kalıtımın yanlış kurulması yüzünden sarpa sarar.
- Ölçütler sonucu belirler
  - Toplam bağlaşıma göre bir tasarım, maksimum bağlaşıma göre öbürü üstün çıkabilir. Kullanılan ölçüt değişince ölçüm sonucu değişir.
  - En doğru ölçü problem alanıdır: varlıklar arasındaki ilişki gerçekte nasılsa tasarım da öyle olmalıdır.
- Uyum ile bağlaşım çeliştiğinde uyum tarafı seçilir
  - Her problemin "olması gereken" bir bağlaşım düzeyi vardır. Bağlaşımı bu düzeyin altına indirmek iyi bir iş değildir.
- "Akıllı insan bir işi bir kez yapar"
  - Bir değişiklik için birden fazla yerde düzeltme yapmak gerekiyorsa yanlış bir tasarım kararı verilmiştir.
- Okunabilirlik
  - Bir alanın ya da metodun açık ve net adlandırılması bile bir refactoring eylemi sayılacaktır.

Kısa Tekrar Notları

- Ders: Kısım 1 GoF tasarım kalıpları, Kısım 2 code smells ve refactoring (Fowler).
- Refactoring = tasarım hatasını geriye doğru uyumluluğu koruyarak düzeltmek.
- İyi tasarım: problem hangi düzeyde parçalanıyorsa tasarım da o düzeyde parçalanır.
- Dış kalite kullanıcıya, iç kalite programcıya yöneliktir.
- Yeniden kullanılabilir kod ek çaba ister; hocanın aktardığı çalışmalara göre kazanç en fazla %30 civarındadır.

- Projelerin katı başarı oranı yaklaşık %50, zorlanılmış başarıyla %80'ler.
- Bağlaşım: diğer parçalarla ilişkide olma oranı. Bağımlılık: hizmetinden yararlanma, iç yapısından haberdar olma, çalışma prensiplerinden haberdar olma, kalıtım.
- Düşük bağlaşım: yeniden kullanılabilirlik ve değişikliğin daha az yan etkisi.
- Uyum: sorumlulukların birbirine yakın olması. Yüksek uyum: anlaşılma, yeniden kullanım, bakım kolaylığı; değişiklikten etkilenme olasılığı düşer.
- Düşük bağlaşım ile yüksek uyum genelde birlikte gelir.
- Uyum ölçüsü: metot-alan kullanım kenarı sayısı / (metot sayısı × alan sayısı). 7/20 = 0,35; bölünce 5/9 ≈ 0,56 ve 2/2 = 1; ortalama ≈ 0,78.
- Dolaylı bağlaşım (A→B→C ise A→C) yarım sayılabilir.
- Veritabanı işlemlerini ayrı sınıfa vermek: bağlaşım 2 → 3,5, ama uyum yüksek ve katmanlı çözüm; doğru çözüm budur.
- UML: düz çizgi + açık ok = sahiplik (association), kesikli çizgi = sahip olmadan kullanma, * = sıfır ya da daha fazla, `1..*` = en az bir, `-` = private, «…» = stereotip.

Detaylı Açıklamalar

Dersin yaklaşımı

Ders, nesneye dayalı tasarımın nasıl iyi yapılacağını iki yönden ele alır. Birinci kısımda Gang of Four'un klasik tasarım kalıpları işlenir; bunlar doğru çözümü gösterir. İkinci kısımda kod kusurları ve refactoring işlenir; burada nesneye yönelimin yanlış kullanım örnekleri adlandırılır ve bu hataların geriye doğru uyumluluk korunarak nasıl düzeltileceği gösterilir. Kalıplar anlatılırken her kalıp önce "kafadan" bir çözümle karşılaştırılır: bu çözüm bugünkü gereksinimi karşılar, ama küçük bir değişiklik istendiğinde işler karışır. Sonra kalıp uygulanır ve kazanımlar ile bedeller birlikte tartışılır. Hocanın deyişiyle hiçbir çözüm sihirli değnek değildir; seçim, yakın zamanda beklenen değişikliklere göre yapılır.

Kod kusurlarına örnek olarak yinelenen kod (Duplicate Code) verildi: birbirine çok benzeyen bir kodun birden fazla yerde bulunması kopyala-yapıştır programlamadan kaynaklanır ve bir bakım kâbusu yaratır. Böyle bir kusur için en uygun refactoring eylemi seçilir; örneğin Extract Method ya da Move Method.

Ön koşul olarak nesneye yönelik programlama ve UML sınıf şemaları gerekir. UML (Unified Modeling Language), birden fazla nesneye yönelik şematik gösterim ekolünün güçlü yanları birleştirilerek oluşturulmuş ve yazılım sektöründe fiilen (de facto) standart olmuş teknik dokümantasyon dilidir. Gang of Four kitabı UML'den önce yazıldığı için şemaları eski gösterimdedir; derste ilk birkaç şemada farklar gösterilecek ve güncel gösterim kullanılacaktır.

Yazılım kalitesi

Amaç iyi tasarım yoluyla kaliteli yazılıma ulaşmaktır. Hocanın sevdiği tanıma göre problem alanı hangi düzeyde parçalara ayrılabiliyorsa, o düzeyde parçalara ayrılabilen tasarım iyidir. Tanımın zayıf yanı, bu "düzeyin" nasıl ölçüleceğinin belirsiz olmasıdır.

Kalite ölçütleri dış ve iç olarak gruplanır. Dış kalite (doğruluk, sistem kaynaklarının etkin kullanımı, kullanım kolaylığı) kullanıcının işini kolaylaştırır. İç kalite geliştiricinin işini kolaylaştırır:

- Yeniden kullanılabilirlik: yazılan kodun başka projelerde de kullanılması. Hoca, çalışmalarda kazancın en fazla %30 civarında görüldüğünü aktardı; çünkü yeniden kullanılabilir kod yazmak, kısa ve orta vadedeki değişiklikleri de düşünmeyi, daha ayrıntılı sınamayı, yani ek çaba ve masrafı gerektirir. Tasarım kalıpları da kurulumda ek çaba ister, bu çaba bakım aşamasında fazlasıyla geri döner.
- İlgi alanlarının ayrılması: problemin ilgi alanları ne düzeyde ayrılıyorsa tasarımın da aynı düzeyde sorumluluklara bölünmesi.
- Esneklik: tasarımın benzer başka problemlere kolayca uyarlanabilmesi.
- Taşınabilirlik: bir platform için yazılan yazılımın başka platformda çalışabilmesi. Hocaya göre bazı dillerin doğrudan derlenmek yerine sanal makinede yorumlanacak şekilde tasarlanmasının ana nedeni budur.
- Okunabilirlik ve anlaşılabilirlik: yazılım projesindeki çabanın rahatça yarısı bakımda harcanır ve okunamayan kodun bakımı yapılamaz.

- Sınanabilirlik: bir sorunu sınamak için birçok farklı yere bakmak gerekiyorsa sorumluluklar büyük olasılıkla doğru atanmamıştır.
- Bakım kolaylığı.

Projelerin başarısı

Katı tanımla bir proje; (1) belirlenen gereksinimleri, (2) belirlenen sürede, (3) belirlenen bütçe içinde karşılarsa başarılıdır. Bu tanımla başarı oranı yaklaşık %50'dir. Üç ayaktan birinde sapmaya izin veren "zorlanılmış" başarı tanımıyla oran yükselir. Hoca, Dobb's Journal 2013 IT Project Success Survey'den bir grafik gösterdi (ekrandan, 00:46:40): geleneksel (şelale) projelerde başarılı oran %50'ye yakın, zorlanılmış ile birlikte yaklaşık %81 (hocanın okuması: başarısız %18, zorlanılmış %32). Belirli bir yöntem izlenmeyen (ad-hoc) projeler gelenekselle hemen hemen aynıdır; çevik (agile), yinelemeli (iterative) ve yalın (lean) süreçlerde başarılı oranı belirgin biçimde yüksektir.

Ulusal bir çalışmada (UYMS 2014) kurumlardaki çevik projelerin başarı oranı soruldu (ekrandan, 00:50:00). Yanıtların dağılımı: başarı oranı %10'un altında 2 (%6), %10-%25 arası 4 (%12), %26-%50 arası 5 (%14), %50-%75 arası 13 (%37), %76-%100 arası 11 (%31). Toplam 35 yanıtın 24'ü (yaklaşık %69) başarı oranını %50'nin üstünde bildirmiştir.

Hocaya göre ilerleme yavaş görünür, çünkü kullanıcılar yazılımın faydasını gördükçe daha büyük problemleri yazılımla çözmeye girişir ve çıta sürekli yükselir. Bu yüzden işin başında adımları sağlam atmak, tasarım kalıplarını bilmek ve ileride bakım sorunu çıkaracak kod kusurlarına düşmemek önemlidir.

Bağlaşım

Bağlaşım, bir parçanın diğer parçalarla ilişkide olma oranıdır. Parça sınıf, alt sistem ya da paket olabilir; derste sınıf düzeyinde çalışılır. Bir sınıf diğerinin hizmetinden yararlanıyorsa, iç yapısından ya da çalışma prensiplerinden haberdarsa ya da onunla kalıtım ilişkisindeyse ona bağımlıdır (ekrandan, 01:07:30). Düşük bağlaşım yeniden kullanılabilirliği artırır ve bir sınıftaki değişikliğin diğer sınıflarda yan etki çıkarma olasılığını düşürür.

Hoca örneği Violet UML editöründe çizdi (ekrandan, 01:20:00). Tasarım I'de A→B ve B→C, tasarım II'de A→B ve A→C vardır:

```
Tasarım I (zincir)          Tasarım II (yıldız)
bağ(A) = 1                  bağ(A) = 2
bağ(B) = 1                  bağ(B) = 0
bağ(C) = 0                  bağ(C) = 0
topBağ = 2, maxBağ = 1      topBağ = 2, maxBağ = 2
```

Toplam bağlaşım eşittir. Tasarım II'de B ve C'nin hiçbir bağımlılığı olmadığı için tek başlarına başka projelerde kullanılabilirler; tasarım I'de B'yi kullanmak isteyen C'yi de almak zorundadır. Maksimum bağlaşım ölçütüne (en kötü durumu en aza indiren karar stratejisi gibi) bakılırsa tasarım I daha iyidir. Dolaylı bağlaşım hesaba katılıp A→C yarım sayılırsa tasarım I için bağ(A) = 1,5, topBağ = 2,5, maxBağ = 1,5 olur (ekrandan, 01:23:40). Bu durumda toplamda II, maksimumda yine I üstündür. Sonuç: sorunun net bir yanıtı yoktur; asıl ölçü, A, B, C varlıklarının problem alanındaki gerçek ilişkisidir.

Uyum ve uyumun ölçülmesi

Uyum, bir parçanın sorumluluklarının birbiriyle uyumlu olma oranıdır (ekrandan, 01:30:00). Yüksek uyumlu sınıf tek bir probleme odaklanmış ve uzmanlaşmıştır; kolay anlaşılır, yeniden kullanılır, bakımı kolaydır ve bir gereksinim değiştiğinde bundan etkilenme olasılığı düşüktür. Düşük bağlaşım ile yüksek uyum genelde birbirini getirir; kötü yönde de durum aynıdır.

Uyumu ölçmek bağlaşımdan zordur. Literatürde statik kod analizine dayanan ölçütler ve kaynak koddaki yorumları dilbilimsel olarak inceleyen (doğal dil işleme, makine öğrenmesi) ölçütler vardır; hoca, ikinci grubun başarısının birinciden çok farklı olmadığını aktardı. Kod analizine dayalı ölçütlerin çoğu, metotların alanları kullanma yoğunluğunun uyumla doğru orantılı olduğu fikrine dayanır (ekrandan, 01:40:00):

- Alanlar a1-a4, metotlar m1-m5. m1: a1, a2; m2: a2; m3: a2, a3; m4: a4; m5: a4. Toplam 7 kenar.
- Tam bağlı graf 4 × 5 = 20 kenar olurdu. Uyum = 7/20 = 0,35.
- Graf ikiye bölünebiliyor: {a1, a2, a3, m1, m2, m3} ve {a4, m4, m5}. Birincisi 5/9 ≈ 0,56, ikincisi 2/2 = 1,00. Ortalama ≈ 0,78.

Grafın hiçbir kenar kesilmeden ikiye bölünebilmesi, sınıfa birbirinden uzak iki sorumluluk yüklendiğini gösterir. Sınıfı ikiye bölmek bir refactoring eylemidir ve uyumu belirgin biçimde artırır.

> Ders sonrası not: Hoca ekrandaki kutulara uyum değerlerini "Bağ=7/20", "Ort-Bağ=0,78" diye yazdı ve sözlü olarak da "bağlaşım" dedi; kendisi de ardından bunun dil sürçmesi olduğunu belirtti. Bu değerler uyum (cohesion) değerleridir.

Veritabanı sorumluluğu örneği

Tasarım I seçilmiş olsun (A→B→C). Varlıklar bir veritabanında saklanacaktır (ekrandan, 01:50:00). Birinci seçenekte her sınıf kendini veritabanına yazar ve okur (A, B, C'den «Database»'e «DB ops» kesikli ilişkileri). İkinci seçenekte bu işi yeni bir D (DbOps) sınıfı yapar; D, A, B ve C'yi sahip olmadan kullanır (kesikli ok). Kullanma ilişkileri yarım sayılırsa bağ(D) = 1,5, topBağ = 3,5, maxBağ = 1,5 olur; toplam bağlaşım 2'den 3,5'e, yani %75 artar. Buna karşın birinci seçenekte uyum düşüktür: veritabanı değişirse (örneğin lisans maliyeti yüzünden Microsoft SQL Server'dan MySQL ya da PostgreSQL'e geçilirse) aynı değişiklik bütün sınıflarda tekrarlanır. İkinci seçenekte yalnız D değişir. Bu yüzden doğru çözüm ikincisidir; ayrıca katmanlı bir çözümdür: A, B, C iş mantığı katmanını, D veritabanı katmanını oluşturur.

UML gösterimi

Derste kullanılan gösterimler: düz çizgi ve ucu açık ok sahiplik (association) ilişkisidir, yani A'da B türünden bir üye alan vardır. B tarafındaki * sıfır ya da daha fazla (A'da B dizisi ya da listesi), `1..*` en az bir anlamına gelir; nicelik yazılmamışsa iki taraf da 1 kabul edilir. - private demektir. Kesikli çizgi sahip olmadan kullanmayı (bir metot içinde geçici nesne kullanma gibi) gösterir. Bir şema bileşeni varsayılan anlamı dışında kullanılırsa «…» stereotip işaretiyle anlamı belirtilir (örneğin «dolaylıBağ», «DB ops»).

Tasarım kalıbı terimi ve kaynaklar

"Tasarım kalıbı" terimi mimarlıktan gelir. Hocanın verdiği mimari örnek: içinde en az bir kişinin bir saatten fazla zaman geçireceği her odanın güneş ışığı alan bir penceresi olmalıdır. Yazılımda da tasarım kalıpları sık karşılaşılan modelleme problemlerinin iyi çözümüne yönelik önerilerdir. Kalıplar yaratımsal, yapısal ve davranışsal olarak üçe ayrılır. Hoca IEEE Xplore'da "Do Structural Design Patterns Promote Design Stability?" (COMPSAC'06) makalesini örnek gösterdi (ekrandan, 02:20:00): bir tasarım, bakım sürecinde ne kadar az değişikliğe uğruyorsa o kadar kararlıdır (stabil). Makalede incelenen Adapter, Bridge, Composite ve Facade ilerleyen haftalarda tek tek işlenecek kalıplardır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.